Dealing with DFAs through actions

Soham Saha
https://soham-saha.github.io
https://soham-saha.codeberg.page

Consider the following question:

> Given that L is regular, show that $\mathrm{sqrt}(L) := \{x \in \Sigma^* : x^2 \in L\}$ is regular.

For this question, suppose we have a complete DFA, $\mathcal{A} = \langle \Sigma, Q, \delta, q_0, F \rangle$.

> Note that $\hat{\delta}(w, -)$ is the same as $\hat{\delta}(w)$, the first one just assumes that we are working with the uncurried version of $\hat{\delta}$.
>
> Note that $\hat{\delta}(w)$ is of type Q^Q, when $w \in \Sigma^*$.

One solution is to consider the set of states Q^3, mark $\{(q_0, a, a) : a \in Q\}$ as initial, and set up transitions so that when we are done reading w starting from any initial state (q_0, a, a), we end up at $\left(\hat{\delta}(w, q_0), a, \hat{\delta}(w, a)\right)$. We mark $\{(x, x, y) : x, y \in Q\}$ as accept states.

The idea can be modified slightly by also remembering where the first word reading started from, which is always q_0. So now, states are Q^4, initial are $\{(q_0, q_0, a, a) : a \in Q\}$, transitions set up such that reading w from some initial (q_0, q_0, a, a) leads us to $\left(q_0, \hat{\delta}(w, q_0), a, \hat{\delta}(w, a)\right)$ and $\{(q_0, x, x, y) : x, y \in Q\}$ are accept states.

We can *very* easily extend this idea to $\mathrm{cbrt}(L) := \{x \in \Sigma^* : x^3 \in L\}$ by considering Q^6 and transitions which map (x, x, y, y, z, z) to $\left(x, \hat{\delta}(w, x), y, \hat{\delta}(w, y), z, \hat{\delta}(w, z)\right)$.

So basically we are storing the graph of the function $\hat{\delta}(w)$ on n many states when we want to take the nth root. Observe that

$$\hat{\delta} : \Sigma^* \to Q^Q$$

is a monoid homomorphism, so $\hat{\delta}$ is the action of the free monoid Σ^* on the set Q, and $\hat{\delta}(w)$ can be interpreted as the action of w on Q.

So why should we stick to just storing the action data for n many states? Why not just do it for all states?

So, in general, we can set up a DFA such that the states are Q^Q, the initial state as $\mathrm{id} : Q^Q$, and transitions such that on reading the word w starting from id we reach $\hat{\delta}(w)$. We are compressing down the (infinite) data set of all words $w \in \Sigma^*$ to the finite set of their actions on Q.

An example of this idea being used in the wild is given below in the `squash` example.

For $K, L \subseteq \Sigma^*$, define $\texttt{squash}(L, K)$ as follows:

$$\texttt{squash}(L, K) = \{uv \mid \exists w \in K, uwv \in L\}$$

6. If K and L are recognizable, then $\texttt{squash}(L, K)$ is also recognizable. Given a complete DFA A_L for L and a complete DFA A_K for K, describe how to get an ϵ-NFA for $\texttt{squash}(L, K)$. *(3 marks)*

Recall the `squash` question above from Quiz 1.

Suppose $\mathcal{A}_L = \langle \Sigma, Q, \delta, q_0, F \rangle$ is a complete DFA which accepts L.

Note that $\hat{\delta} : \Sigma^* \to Q \to Q$, so $\hat{\delta}(w)$

The idea is that as we read $x \in \Sigma^*$, we will be storing all possible splittings of it as (u, v) where $x = uv$. But because we cannot store infinitary data (infinitary in the sense that we cannot store for *all* $x \in \Sigma^*$), and here anyway we don't *need* the data of the word itself, we will be storing $\left\{\left(\hat{\delta}(u), \hat{\delta}(v)\right) : u, v \in \Sigma^* \text{ s.t. } x = uv\right\}$. Observe that we will accept x whenever there are $u, v \in \Sigma^*$ such that $uv = x$ and $\left[\exists w \in K \text{ s.t. } \left(\hat{\delta}(v) \circ \hat{\delta}(w) \circ \hat{\delta}(u)\right)(q_0) \in F\right]$.

So, to design a DFA for `squash(L,K)`, we set

- States $:= \mathcal{P}\left(Q^Q \times Q^Q\right)$
- Transition := On reading $a \in \Sigma$, transition from $S \in \mathcal{P}\left(Q^Q \times Q^Q\right)$ to

$$\{(f, \delta(a) \circ g) : (f, g \in S)\} \cup \{(\delta(a) \circ g \circ f, \text{id}) : (f, g) \in S\}$$

- Initial state $:= \{(\text{id}, \text{id})\}$
- Final states :=

$$S \in \mathcal{P}\left(Q^Q \times Q^Q\right) \text{ is final} \Leftrightarrow \left\{(f, g) : \exists w \in K \text{ s.t } \left(g \circ \hat{\delta}(w) \circ f\right)(q_0) \in F\right\} \cap S \neq \varphi$$

The semantics that we are targeting here is that on reading the word $x \in \Sigma^*$ starting from $\{(\text{id}, \text{id})\}$, we end up at the set $\left\{\left(\hat{\delta}(u), \hat{\delta}(v)\right) : uv = x\right\}$.

This DFA accepts `squash(L,K)`.

Note that we never used that K is regular. We would need that K is regular to actually construct a DFA, the above only shows the existence of a DFA.

But, observe that the accept condition is *non-algorithmic*, in the sense that given $S \in P\left(Q^Q \times Q^Q\right)$, we cannot easily decide whether S should be an accept state or not, using the above definition of final states. Can we somehow make this constructive?

If we can constructively produce the finite set $\left\{\hat{\delta}(w) : w \in K\right\}$ then we would have an algorithm to check whether $S \in \mathcal{P}\left(Q^Q \times Q^Q\right)$ should be accepting or not.

We already have the DFA $\mathcal{A}_L = \langle \Sigma, Q_L, \delta_L, q_{0_L}, F_L \rangle$ for L.

Suppose K is accepted by $\mathcal{A}_K = \langle \Sigma, Q_K, \delta_K, q_{0_K}, F_K \rangle$.

Consider the directed graph on $Q_L^{Q_L} \times Q_K$, where edges are

$$E \coloneqq \{(f, x), (\delta_L(a) \circ f, \delta_K(a, x)) : a \in \Sigma\}$$

Observe that for arbitrary $S \in Q^Q$,

$$S \in \left\{\hat{\delta}_L(w) : w \in K\right\} \Leftrightarrow \exists k \in F_K \text{ s.t. } (S, k) \text{ is reachable from } \left(\mathrm{id}, q_{0_K}\right)$$

This gives an algorithm for constructing $\left\{\hat{\delta}_L(w) : w \in K\right\}$.

One of the key ideas behind this solution style is the observation that

$$a_1 a_2 ... a_k \in L \Leftrightarrow \left(\hat{\delta}(a_k) \circ \hat{\delta}(a_{k-1}) \circ ... \circ \hat{\delta}(a_1)\right)(q_0) \in F$$

whenever L is regular and has complete DFA $\langle \Sigma, Q, \delta, q_0, F \rangle$.

So, we don't have to carry around the data of $a_1 a_2 ... a_k$ as we traverse the DFA we are going to make, it is enough to just store their actions.